

EJERCICIOS DE BUCLES EN BASH

Ejercicios con **for**.

*En los ejercicios 1 y 2 se muestra como se usan los comandos: `chmod +x nombreArchivo.sh` para dar los permisos necesarios a cada archivo, y `./nombreArchivo.sh` para mostrar el resultado en pantalla. Esto se hace en TODOS los ejercicios pero por razones obvias no pongo las capturas en cada ejercicio.

- 1) Escribe un script que use un bucle **for** para imprimir los números del 1 al 10.

```
usuario@ubuntu: ~  
usuario@ubuntu:~$ touch efor1.sh  
usuario@ubuntu:~$ nano efor1.sh  
  
GNU nano 7.2 efor1.sh  
#!/usr/bin/env bash  
  
for num in {1..10} # SE INICIA EL BUCLE CON LA VARIABLE num QUE TOMA EL VALOR EN CADA ITERACIÓN DENTRO DEL RANGO {1..10}.  
do  
    echo $num # MUESTRA EL VALOR ACTUAL DE LA VARIABLE EN ESA ITERACIÓN.  
done  
  
usuario@ubuntu:~$ chmod +x efor1.sh  
usuario@ubuntu:~$ ./efor1.sh  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
usuario@ubuntu:~$
```

- 2) Crea un script que defina una variable con tres nombres de usuarios (ej: `users="ana ben carlos"`) y use un bucle **for** para imprimir un saludo para cada uno.

```
usuario@ubuntu: ~  
usuario@ubuntu:~$ touch efor2.sh  
usuario@ubuntu:~$ nano efor2.sh  
  
GNU nano 7.2 efor2.sh  
#!/usr/bin/env bash  
  
nombres="Ana Ben Carlos" # SE DEFINE LA VARIABLE nombres LA CUAL CONTIENE UNA CADENA CON TRES NOMBRES SEPARADOS POR ESPACIOS.  
for nombre in ${nombres} # SE INICIA EL BUCLE FOR DONDE CADA nombre SEPARADO POR ESPACIO SE TRATARÁ COMO UN ELEMENTO DISTINTO.  
do  
    echo "Hola ${nombre}" # SE IMPRIMIRAN 3 LÍNEAS (UNA POR CADA nombre DENTRO DE LA VARIABLE nombres).  
done  
  
usuario@ubuntu:~$ chmod +x efor2.sh  
usuario@ubuntu:~$ ./efor2.sh  
Hola Ana  
Hola Ben  
Hola Carlos
```

- 3) Usa un bucle **for** para imprimir los números pares del 2 al 20.

```
GNU nano 7.2 efor3.sh  
#!/usr/bin/env bash  
  
for num in {2..20..2} # {2..20..2} EMPIEZA EN 2, TERMINA EN 20, AVANZA DE 2 EN 2. EN CADA ITERACIÓN, num TOMA UNO DE ESOS VALORES.  
do  
    echo $num # echo LO IMPRIME.  
done
```

Opción 2:

```
GNU nano 7.2 efor3.sh  
#!/usr/bin/env bash  
  
for num in {1..20}  
do  
    if [[ $((num % 2)) -eq 0 ]] # $((num % 2)) CALCULA EL RESTO DE DIVIDIR ENTRE 2. SI EL RESTO ES 0, EL NÚMERO ES PAR. SOLO ENTONCES SE IMPRIME.  
    then  
        echo $num  
    fi  
done
```

- 4) Define un *array* de 4 frutas (ej: `frutas=( "Manzana" "Banana" "Naranja" "Uva" )`) y usa un bucle **for** para imprimir cada elemento del array.

```
GNU nano 7.2 efor4.sh
#!/usr/bin/env bash

# DEFINIMOS EL ARRAY
frutas=( "Manzana" "Banana" "Naranja" "Uva" ) # frutas=(...) CREA EL ARRAY.

# RECORREMOS EL ARRAY
for fruta in "${frutas[@]}" # "${frutas[@]}" OBTIENE TODOS LOS ELEMENTOS DEL ARRAY. EN CADA ITERACION, LA VARIABLE fruta TOMA UNO DE LOS VALORES.
do
    echo "$fruta" #echo LO IMPRIME.
done
```

- 5) Escribe un script que use un bucle **for** para mostrar la tabla de multiplicar del 3 (del 3x1 al 3x10).

```
GNU nano 7.2 efor5.sh
#!/usr/bin/env bash

for i in {1..10} # {1..10} GENERA LOS NÚMEROS DEL 1 AL 10. EN CADA ITERACIÓN, i TOMA UNO DE ESOS VALORES.
do
    resultado=$((3 * i)) # $((3 * i)) REALIZA LA OPERACIÓN ARITMÉTICA.
    echo "3 x $i = $resultado" # MUESTRA EL RESULTADO.
done
```

Ejercicios con *while*. (*while* → *ejecuta mientras la condición sea verdadera.*)

- 6) Escribe un script que use un bucle **while** para imprimir los números del 1 al 5.

```
GNU nano 7.2 ewhile1.sh
#!/usr/bin/env bash

contador=1 # PUNTO DE INICIO.

while [[ $contador -le 5 ]] # LA CONDICIÓN SIGNIFICA: MIENTRAS SEA MENOR O IGUAL QUE 5.
do
    echo $contador # IMPRIME EL VALOR ACTUAL.
    ((contador++)) # INCREMENTA EL CONTADOR.
done
```

- 7) Crea un script que use un bucle **while** para contar hacia atrás, desde 5 hasta 1.

```
GNU nano 7.2 ewhile2.sh
#!/usr/bin/env bash

contador=5 # PUNTO DE INICIO.

while [[ $contador -ge 1 ]] # MIENTRAS CONTADOR SEA MAYOR O IGUAL QUE 1.
do
    echo $contador # IMPRIME EL VALOR ACTUAL.
    ((contador--)) # CONTADOR SE DECREMENTA EN 1 EN CADA VUELTA.
done
```

- 8) Escribe un script que pida al usuario su nombre. Usa un bucle **while** para seguir preguntando si el usuario no introduce ningún texto (cadena vacía).

```
GNU nano 7.2 ewhile3.sh *
#!/usr/bin/env bash

read -p "¿Cuál es tu nombre?" nombre # read -p MUESTRA EL MENSAJE Y GUARDA A ENTRADA EN NOMBRE.

while [[ -z $nombre ]] # [[ -z $nombre ]] COMPRUEBA SI LA VARIABLE ESTA VACÍA.
do
    echo "El nombre no puede estar vacío." # MIENTRAS ESTÉ VACÍA, MUESTRA EL MENSAJE Y SE REPITE EL BUCLE.
    read -p "Introduce tu nombre: " nombre # CUANDO EL USUARIO INTRODUCE TEXTO, EL BUCLE TERMINA.
done

echo "Hola $nombre" # SE MUESTRA EL SALUDO.
```

- 9) Crea un script que use un bucle **while** para sumar los números del 1 al 10 y muestre solo el resultado final (55).

```
GNU nano 7.2 ewhile4.sh
#!/usr/bin/env bash

contador=1 # SE INICIA CONTADOR EN 1.
suma=0 # VARIABLE ACUMULADORA.

while [[ $contador -le 10 ]] # MIENTRAS CONTADOR SEA MENOR O IGUAL QUE 10.
do
    ((suma += contador)) # SE SUMA EL VALOR ACTUAL.
    ((contador++)) # SE INCREMENTA EL CONTADOR.
done

echo $suma
```

- 10) Escribe un bucle **while** que imprima el valor de un contador mientras este sea menor que 3, e

incrementa el contador en 1 en cada iteración.

```
GNU nano 7.2                               ewhile5.sh
#!/usr/bin/env bash

contador=0 # CONTADOR SE INICIA EN 0.

while [[ $contador -lt 3 ]] # [[ $contador -lt 3 ]] SE EJECUTA MIENTRAS CONTADOR SEA MENOR QUE 3.
do
    echo $contador
    ((contador++))
done
```

Ejercicios con *until*. (*until* → *ejecuta mientras la condición sea falsa*.)

- 11) Escribe un script que use un bucle **until** para imprimir los números del 1 al 5.

```
GNU nano 7.2                               euntil1.sh
#!/usr/bin/env bash

contador=1

until [[ $contador -gt 5 ]] # SE EJECUTA MIENTRAS NO SEA MAYOR QUE 5.
do
    echo $contador # IMPRIME EL VALOR ACTUAL.
    ((contador++)) # SE INCREMENTA EL CONTADOR.
done
```

- 12) Crea un script que use un bucle **until** para contar hacia atrás, desde 5 hasta 1.

```
GNU nano 7.2                               euntil2.sh
#!/usr/bin/env bash

contador=5 # SE INICIALIZA UN CONTADOR EN 5.

until [[ $contador -lt 1 ]] # SE EJECUTA EL BUCLE MIENTRAS CONTADOR NO SEA MENOR QUE 1.
do
    echo $contador # IMPRIME EL VALOR ACTUAL.
    ((contador--)) # SE DECREMENTA EL CONTADOR.
done
```

- 13) Escribe un script que pida al usuario que escriba "salir". Usa un bucle **until** para que el script se repita hasta que el usuario escriba exactamente "salir".

```
GNU nano 7.2                               euntil3.sh
#!/usr/bin/env bash

until [[ $respuesta == "salir" ]] # EL BUCLE SE EJECUTA MIENTRAS LA CONDICIÓN SEA FALSA.
do
    read -p "Escribe 'salir' para terminar: " respuesta # MIENTRAS EL USUARIO ESCRIBA CUALQUIER OTRA COSA QUE NO SEA "salir" EL BUCLE SE REPITE.
done

echo "Programa finalizado." # CUANDO SE ESCRIBA "salir" EL BUCLE TERMINA Y SE MUESTRA EL MENSAJE EN PANTALLA.
```

- 14) Define una variable contador en 10. Usa un bucle **until** para imprimir el contador mientras decrece, hasta que el contador sea 0.

```
GNU nano 7.2                               euntil4.sh
#!/usr/bin/env bash

contador=10 # VALOR INICIAL.

until [[ $contador -eq 0 ]] # EL BUCLE SE EJECUTARÁ MIENTRAS contador NO SEA 0.
do
    echo $contador # IMPRIME EL VALOR ACTUAL.
    ((contador--)) # SE DECREMENTA EL CONTADOR.
done
```

- 15) Escribe un script que use un bucle **until** y una expresión de archivo para esperar hasta que un archivo llamado listo.txt exista en el directorio actual. (Puedes probarlo creando el archivo en otra terminal).

```
GNU nano 7.2                               euntil5.sh
#!/usr/bin/env bash

echo "Esperando a que el archivo listo.txt exista..."

until [[ -f "listo.txt" ]] # EJECUTA EL BLOQUE MIENTRAS LA CONDICIÓN SEA FALSA.
do
    sleep 1 # EVITA QUE EL BUCLE CONSUMA CPU CONSTANTEMENTE.
done

echo "El archivo listo.txt ya existe."
```